

Chemfarm for collective quantum optics

Bonus: EM Green's functions

Nikita Leppenen

February 2026

Vectorized 1D Greens function

```
def create_1d_gnm_matrix_vectorized(positions, gamma1D=1):  
    """Creates the Green's function matrix for 1D lattice using vectorized operations."""  
    N = len(positions)  
  
    # Compute all pairwise displacement vectors in 1D  
    r_vec = positions[:, np.newaxis] - positions[np.newaxis, :]  
    # Compute Euclidean distances  
    R = np.abs(r_vec)  
    # Set diagonal elements to avoid division by zero  
    np.fill_diagonal(R, 1.0)  
    # Compute kr  
    kr = 2 * np.pi * R  
    # Compute the 1D Green's function  
    g1 = gamma1D / 2 * np.exp(1j * kr)  
    np.fill_diagonal(g1, 0)  
  
    gnm = 2 * np.real(g1)  
    dnm = np.imag(g1)  
  
    return gnm, dnm
```

3D lattice with numpy

```
#create an 3D array with x,y,z lattice with lattice spacing a_x, a_y, a_z
def create_lattice(nx, ny, nz, a_x, a_y, a_z):
    """Creates a 3D lattice of points with given dimensions and spacings."""
    x = np.arange(-nx/2, nx/2) * a_x
    y = np.arange(-ny/2, ny/2) * a_y
    z = np.arange(-nz/2, nz/2) * a_z
    xv, yv, zv = np.meshgrid(x, y, z, indexing='ij')
    lattice = np.stack((xv, yv, zv), axis=-1).reshape(-1, 3)
    return lattice
```

3D Greens function with numpy

```
def create_gnm_matrix_vectorized(positions):  
    """Creates the Green's function matrix using vectorized operations."""  
    N = len(positions)  
  
    # Compute all pairwise displacement vectors using broadcasting  
    # Shape: (N, N, 3)  
    r_vec = positions[:, np.newaxis, :] - positions[np.newaxis, :, :]  
  
    # Compute Euclidean distances  
    # Shape: (N, N)  
    R = np.linalg.norm(r_vec, axis=2)  
  
    # Set diagonal elements to a non-zero value to avoid division by zero  
    np.fill_diagonal(R, 1.0)  
    kr = 2 * np.pi * R  
  
    # Create e_0 vector (polarization)  
    e_0 = (1 / np.sqrt(2)) * np.array([1, 1j, 0])  
    #e_0 = np.array([0, 0, 1])  
    # Compute dot products between e_0 and all displacement vectors  
    # Shape: (N, N)
```

```

# Shape: (N, N)
dot_products = np.sum(e_0 * r_vec, axis=2)

# Compute cos^2(theta)
# Shape: (N, N)
cos2_theta = np.abs(dot_products / R) ** 2

# Compute the Green's function
g1 = np.exp(1j * kr) / kr * (
    (1 + (1j * kr - 1) / kr**2) + cos2_theta * (-1 + (3 - 3 * 1j * kr) / kr**2))

# Set diagonal elements to zero (no self-interaction)
np.fill_diagonal(g1, 0)

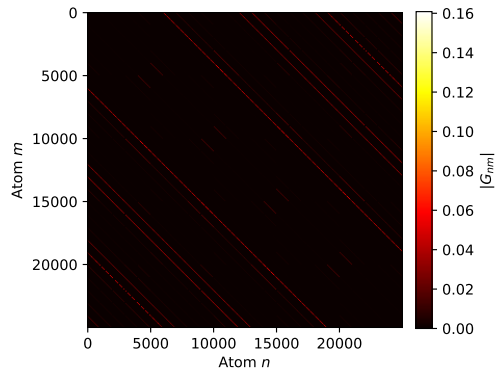
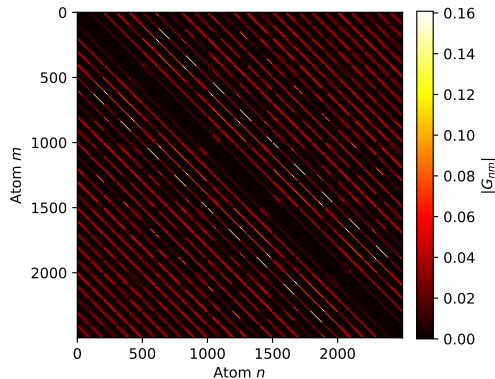
# Compute gnm and dnm
dnm = -3/4 * np.real(g1)
gnm = 3/2 * np.imag(g1)
#dnm = np.zeros((N,N))

return gnm, dnm

```

Example of 3D Greens function

Code: 3DGF.py and $N_z = 1000$ on Chemfarm



$|G_{nm}|$ for a 5x5 lattice in (xy) plane repeated 100 (left) and 1000 (right) times in z direction.
Spacing: $a_x = a_y = 0.5\lambda$ and $a_z = \lambda$. For $N_z = 1000$ – sparse structure is seen – **could be studied!**

Experiments with sparse Green's function

Appearance of 2D electrodynamics

Coherent Δ_{nm} ; incoherent γ_{nm} ; $\frac{\gamma_{nm}}{2} + i\Delta_{nm} = -i\frac{3\gamma}{4} \bar{G}(\mathbf{r}_n - \mathbf{r}_m)$

λ - **spacing along $z \Rightarrow$ system is (almost) translation invariant in z direction.**

3D Green's function (xyz) $\rightarrow$ 2D Green's function (xy - plane) between z - chains of atoms.

1. Chain self interaction (without probe atom $m_z = n_z$):

$$\frac{\tilde{\gamma}_{n_{\perp}, n_{\perp}}}{2} + i\tilde{\Delta}_{n_{\perp}, n_{\perp}} = \sum_{m_z \neq n_z} \left[\frac{\gamma_{n_{\perp}, n_z; n_{\perp}, m_z}}{2} + i\Delta_{n_{\perp}, n_z; n_{\perp}, m_z} \right] = -i\frac{3\gamma}{4} \sum_{m_z \neq n_z} \bar{G}(\mathbf{r}_{n_{\perp}, n_z} - \mathbf{r}_{n_{\perp}, m_z})$$

2. Interaction between different chains:

$$\frac{\tilde{\gamma}_{n_{\perp}, m_{\perp}}}{2} + i\tilde{\Delta}_{n_{\perp}, m_{\perp}} = \sum_{m_z} \left[\frac{\gamma_{n_{\perp}, n_z; m_{\perp}, m_z}}{2} + i\Delta_{n_{\perp}, n_z; m_{\perp}, m_z} \right] = -i\frac{3\gamma}{4} \sum_{m_z \neq n_z} \bar{G}(\mathbf{r}_{n_{\perp}, n_z} - \mathbf{r}_{m_{\perp}, m_z})$$